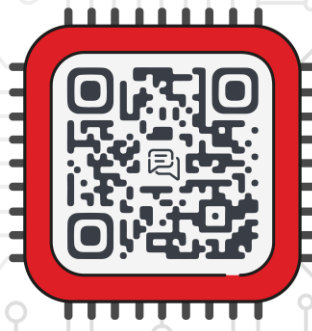




نظري

7

36

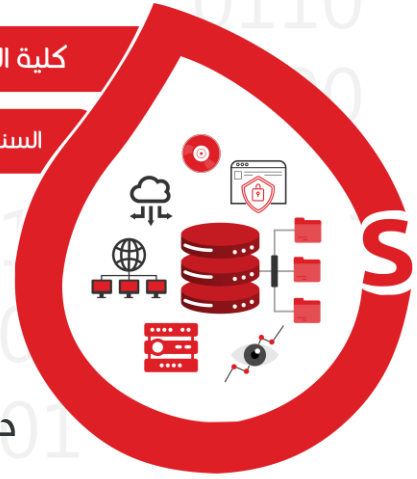
5400 sp

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Query Optimization

د. مادلين عبود



RB Informatics; 23/11/2025

محتوى مجاني غير مخصص للبيع التجاري

قواعد معطيات ٢

في المحاضرتين السابقتين تحدثنا عن تنفيذ الاستعلام Query Processing وفي هذه المحاضرة سنركز على:

تحسين الاستعلام (Query Optimization) وهو عملية اختيار أفضل طريقة لتنفيذ الاستعلام على قاعدة البيانات بحيث يعطي نفس النتيجة ولكن بأقل كلفة زمنية واستهلاك موارد (CPU، ذاكرة، I/O).

الفرق بين Query Optimization و Query Processing

- Query Processing: العملية الكاملة لتنفيذ الاستعلام.
- Query Optimization: مرحلة داخل هذه العملية (داخل Query Processing) تهدف لجعل التنفيذ أسرع وأقل كلفة.

من المهم أن نعلم أن:

- يمكن كتابة الاستعلام بعدة طرق متكافئة منطقيًا.
- جميع هذه الطرق تعطي نفس النتيجة.
- لكن كلفة التنفيذ تختلف من طريقة لأخرى.
- دور ال Query Optimizer هو اختيار أنسب مخطط تنفيذ.

مراحل تحسين الاستعلام:

1. توليد تعابير جبرية متكافئة (Relational Algebra Expressions):

- تحويل الاستعلام إلى عدة تعابير جبرية متكافئة منطقيًا.
- يتم ذلك باستخدام قواعد التكافؤ (Equivalence Rules).

2. تقييم مخططات التنفيذ:

- لكل تعبير مكافئ يتم إنشاء مخطط تنفيذ (Execution Plan).
- يتم حساب الكلفة المتوقعة لكل مخطط.

3. اختيار المخطط الأنسب:

■ يتم اختيار المخطط ذو أقل كلفة.

قواعد التكافؤ (Equivalence Rules)

هي قواعد تسمح بتحويل تعبير جبري إلى تعبير آخر مكافئ له دون تغيير النتيجة.

↪ التحسين (الاختزال المبني على الكلفة) :Cost-Based Optimization:

■ يعتمد على تقدير:

- حجم الجداول
- عدد الصفوف
- وجود فهارس (Indexes).

■ يتم اختيار مخطط التنفيذ بناءً على أقل كلفة متوقعة.

↪ التعابير الجبرية المتكافئة:

تعابير تعطي:

- نفس العلاقات الناتجة
- نفس الصفوف
- نفس الصفات
- مع إمكانية اختلاف:

- ترتيب الصفات
- ترتيب العمليات الداخلية

↪ أهم قواعد التكافؤ (باختصار):

1. تفكيك شرط الاختيار (Cascade of Selection):

إذا كان شرط الاختيار يحتوي على AND:

Conjunctive selection operations can be deconstructed into a sequence of individual selections.

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

■ يسمح بدفع الشروط للأماكن المناسبة لتحسين الأداء.

القوة أن تمضي رغم أنك متعب، لا لأنك مرتاح

2. التبادلية في الاختيار (Commutativity of Selection):

Selection operations are commutative.

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

ترتيب شروط الاختيار لا يؤثر على النتيجة.

3. تبسيط الإسقاط (Cascade of Projection):

Only the last in a sequence of projection operations is needed, the others can be omitted.

$$\Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) = \Pi_{L_1}(E)$$

نحتاج فقط آخر Projection.

4. الاختيار مع ضرب الديكارتى و Join:

ال Theta Join هو ضرب ديكارتى + اختيار:

Selections can be combined with Cartesian products and theta joins.

- a. $\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$
- b. $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) = E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$

5. التبادلية في Join:

5. Theta-join operations (and natural joins) are commutative.

$$E_1 \bowtie_{\theta} E_2 = E_2 \bowtie_{\theta} E_1$$

النتيجة نفسها (مع تجاهل ترتيب الأعمدة).

6. الترابطية في Join (Associativity):

(a) Natural join operations are associative:

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

(b) Theta joins are associative in the following manner:

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 = E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

where θ_2 involves attributes from only E_2 and E_3 .

■ مهمة جداً في إعادة ترتيب ال Join لتحسين الأداء.

7. توزيع الاختيار على Join:

The selection operation distributes over the theta join operation under the following two conditions:

- (a) When all the attributes in θ_0 involve only the attributes of one of the expressions (E_1) being joined.

$$\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$$

- (b) When θ_1 involves only the attributes of E_1 and θ_2 involves only the attributes of E_2 .

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$$

■ قاعدة أساسية لتقليل حجم البيانات مبكراً.

8. توزيع الإسقاط على Join:

The projection operation distributes over the theta join operation as follows:

- (a) if θ involves only attributes from $L_1 \cup L_2$:

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) = (\Pi_{L_1}(E_1)) \bowtie_{\theta} (\Pi_{L_2}(E_2))$$

- (b) Consider a join $E_1 \bowtie_{\theta} E_2$.

- Let L_1 and L_2 be sets of attributes from E_1 and E_2 , respectively.
- Let L_3 be attributes of E_1 that are involved in join condition θ , but are not in $L_1 \cup L_2$, and
- let L_4 be attributes of E_2 that are involved in join condition θ , but are not in $L_1 \cup L_2$.

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) = \Pi_{L_1 \cup L_2}((\Pi_{L_1 \cup L_3}(E_1)) \bowtie_{\theta} (\Pi_{L_2 \cup L_4}(E_2)))$$

■ يسمح بحذف الأعمدة غير الضرورية قبل تنفيذ Join بشرط الحفاظ على أعمدة الربط.

9. عمليات المجموعات – التبادلية:

The set operations union and intersection are commutative

$$E_1 \cup E_2 = E_2 \cup E_1$$

$$E_1 \cap E_2 = E_2 \cap E_1$$

- (set difference is not commutative).

■ الفرق (-) غير تبادلي.

10. عمليات المجموعات – الترابطية:

Set union and intersection are associative.

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$

$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

11. توزيع الاختيار على عمليات المجموعات:

The selection operation distributes over $\cup$, $\cap$ and $-$.

$$\sigma_{\theta}(E_1 - E_2) = \sigma_{\theta}(E_1) - \sigma_{\theta}(E_2)$$

and similarly for $\cup$ and $\cap$ in place of $-$

Also:

$$\sigma_{\theta}(E_1 - E_2) = \sigma_{\theta}(E_1) - E_2$$

and similarly for $\cap$ in place of $-$, but not for $\cup$

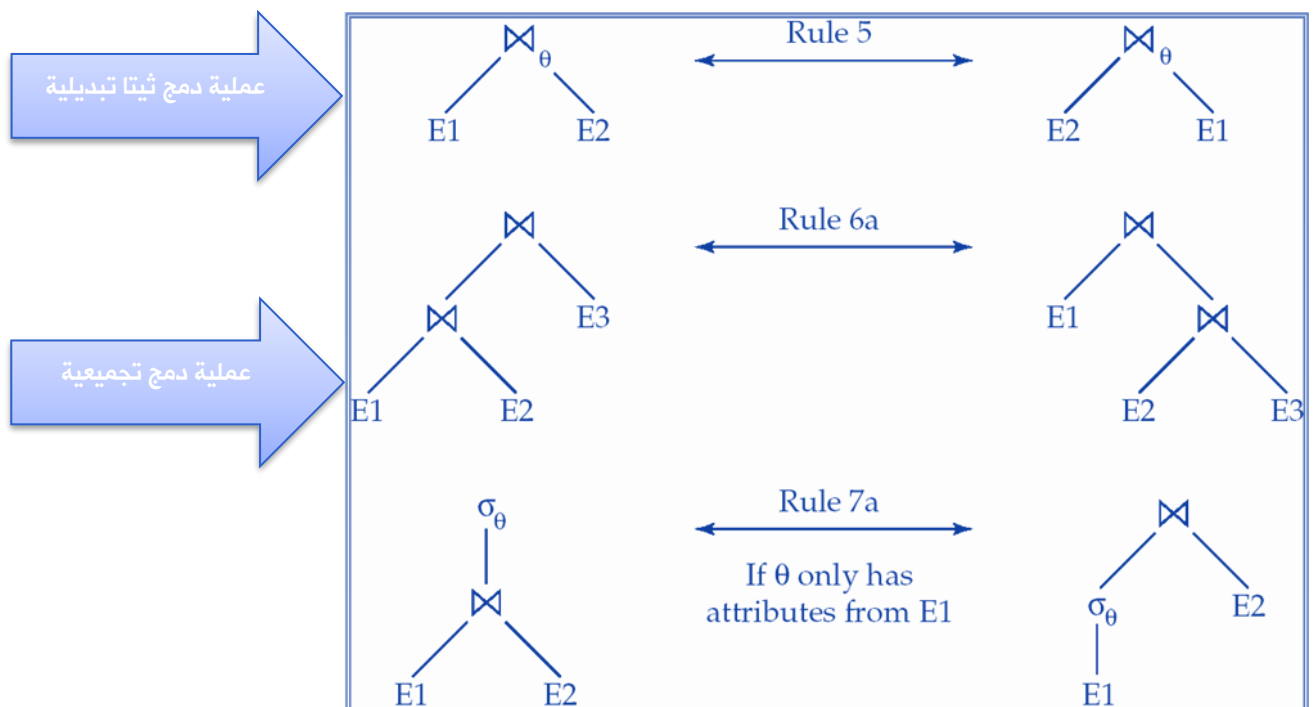
وينطبق أيضاً على $\cup$ و $\cap$ مع بعض القيود.

12. توزيع الإسقاط على Union:

The projection operation distributes over union

$$\Pi_L(E_1 \cup E_2) = (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$

التمثيل البياني لبعض القواعد:



مثال 1: ليكن لدينا الاستعلام التالي

Find the names of all customers who have an account at some branch located in Brooklyn:

أوجد أسماء الزبائن الذين لديهم حساب في فرع يقع في Brooklyn:

باعتبار:

Branch (branch-name, branch-city, assets)

Account (branch-name, account-number, balance)

Depositor (customer-name, account-number)

الصيغة الأصلية (قبل التحسين):

$$\Pi_{customer_name}(\sigma_{branch_city = "Brooklyn"}(branch \bowtie (account \bowtie depositor)))$$

هنا يتم:

1. ربط الجداول branch, account, depositor

2. ثم تطبيق شرط المدينة بعد تنفيذ الـ joins

← هذا يعني معالجة كمية كبيرة من البيانات أولاً.

التحويل باستخدام القاعدة 7a (Pushing Selection):قاعدة 7a تقول:

■ إذا كان شرط الاختيار يعتمد فقط على صفات علاقة واحدة، يمكن دفعه قبل الـ Join.

شرط:

■ branch_city موجود فقط في جدول branch

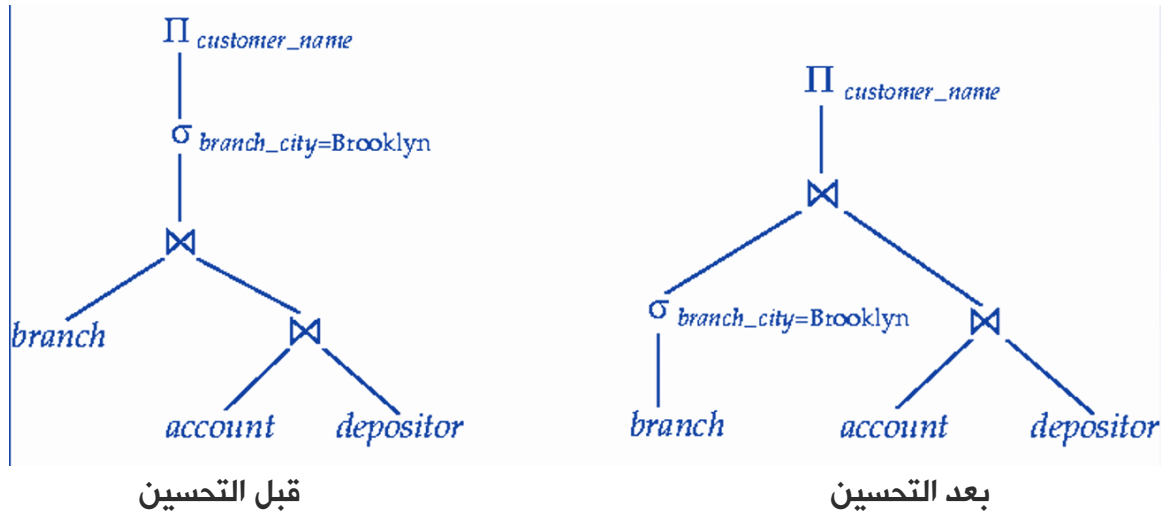
الصيغة المحسنة (بعد التحسين):

$$\Pi_{customer_name}((\sigma_{branch_city = "Brooklyn"}(branch)) \bowtie (account \bowtie depositor))$$
لماذا هذا أفضل؟

- يتم تصفية جدول branch أولاً
- عدد الصفوف في branch يصبح أقل
- الـ join يتم على بيانات أصغر
- انخفاض الكلفة الزمنية واستهلاك الذاكرة

الفكرة الأساسية المكتسبة من هذا الاختزال: نفّذ عمليات (Selection) σ في أقرب وقت ممكن لأنها تقلل حجم

البيانات قبل عمليات الـ Join المكلفة.



مثال 2:

أوجد أسماء الزبائن الذين لديهم حساب في فرع يقع في Brooklyn وكان رصيد الحساب أكبر من \$1000:

الصيغة الأصلية (قبل التحسين):

$$\Pi_{customer_name}(\sigma_{branch_city = "Brooklyn" \wedge balance > 1000} (branch \bowtie (account \bowtie depositor)))$$

يتم هنا:

- تنفيذ جميع ال Join أولاً.
 - ثم تطبيق شرطين معاً بعد ذلك.
- كلفة عالية.

↪ التحويل باستخدام الترابطية في Join (Rule 6a):

نغير ترتيب ال joins:

$$\Pi_{customer_name}((\sigma_{branch_city = "Brooklyn" \wedge balance > 1000} (branch \bowtie account)) \bowtie depositor)$$

- النتيجة نفسها لكن أصبح لدينا فرصة لتحسين إضافي.

↪ دفع عمليات الاختيار (Rule 7):

نلاحظ:

- branch_city يخص branch.
- balance يخص account.

نقسم شرط الاختيار:

$$\sigma_{branch_city = "Brooklyn"}(branch)$$

$$\sigma_{balance > 1000}(account)$$

$$\sigma_{branch_city = "Brooklyn"} (branch) \bowtie \sigma_{balance > 1000} (account)$$

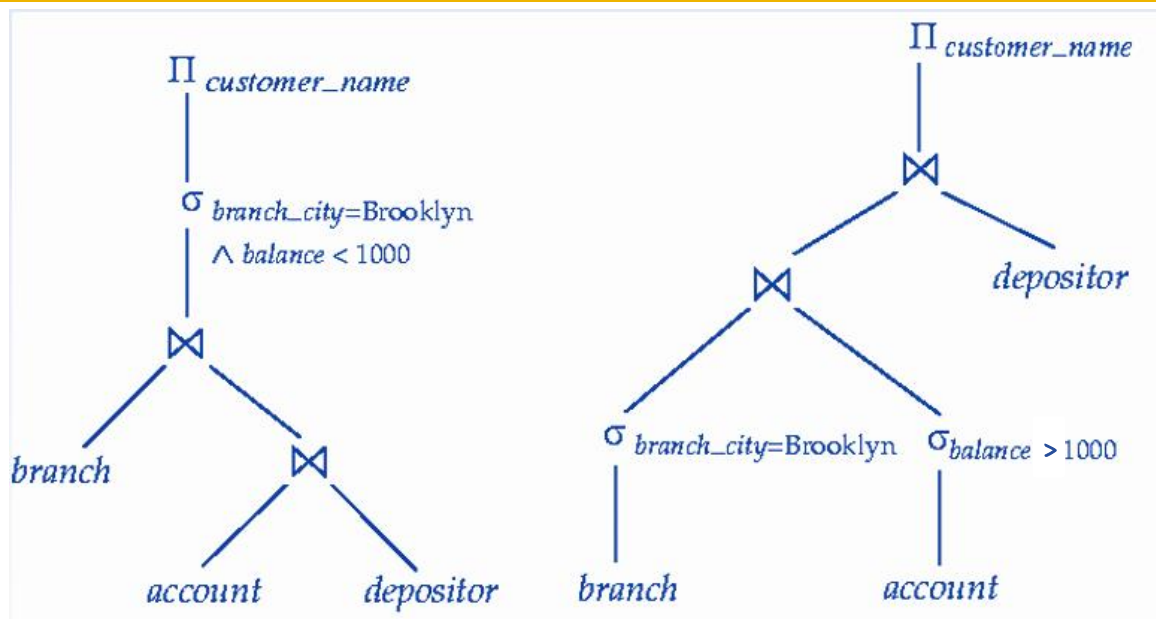
لماذا هذا التحسين مهم؟

- تقليل حجم جدول branch مبكراً.
- تقليل حجم جدول account مبكراً.
- عمليات join تتم على بيانات أصغر
- ← تحسين كبير في الأداء.

في هذا المثال استخدمنا أكثر من قاعدة تكافؤ:

- Rule 6a: Associativity of Join
- Rule 7: Pushing Selections

نستنتج انه يمكن لسلسلة من التحويلات ان تعطي نفس النتيجة ولكن بكلفة أقل.



قبل التحسين

بعد التحسين

مثال 3:

التعبير قبل التحسين:

$$\Pi_{customer_name}((\sigma_{branch_city = "Brooklyn"} (branch) \bowtie account) \bowtie depositor)$$

عند التنفيذ:

$$(\sigma_{branch_city = "Brooklyn"} (branch) \bowtie account)$$

نحصل على علاقة تحتوي الصفات:

(branch_name, branch_city, assets, account_number, balance)

ولكن كثير من هذه الصفات غير مستخدمة لاحقاً.

ما الذي نحتاجه فعلياً؟

- ال Join مع depositor نحتاج فقط: account_number
 - للنتيجة النهائية نحتاج: customer_name من (depositor)
- إذا بقي الصفات يمكن حذفها مبكراً.

تطبيق قواعد التكافؤ 8a و b8:

قواعد Pushing Projections تسمح بدفع عملية الإسقاط قبل ال join مع الإبقاء على صفات الربط فقط.

التعبير بعد التحسين:

$$\Pi_{customer_name} ((\Pi_{account_number} ((\sigma_{branch_city = "Brooklyn"} (branch) \bowtie account)) \bowtie depositor))$$

لماذا هذا التحسين فعال؟

- حذف الأعمدة غير الضرورية مبكراً
- تقليل حجم السجلات (عرض الصفوف)
- تقليل كلفة عمليات ال join
- تحسين الأداء العام للاستعلام

في هذه المثال قمنا بـ:

حذف الأعمدة غير المطلوبة بأقرب وقت = Pushing Projections
مع الحفاظ على:

- صفات ال join
- صفات النتيجة النهائية فقط.

ترتيب عمليات الدمج Join Ordering:

بسبب خاصية الترابطية (Associativity) في Join:
فإن:

$$(r1 \bowtie r2) \bowtie r3 = r1 \bowtie (r2 \bowtie r3)$$

- يمكن تغيير ترتيب تنفيذ ال joins دون تغيير النتيجة لكن الكلفة تختلف بشكل كبير.
- إذا كان ناتج $(r2 \bowtie r3)$ كبير نسبياً وناتج $(r1 \bowtie r2)$ صغير نسبياً، فالأفضل تنفيذ العملية كالتالي:

$$(r1 \bowtie r2) \bowtie r3$$

إذا المبدأ هنا يقول:

ابدأ بربط العلاقات الأصغر أو الأكثر تقييداً حتى لا تنتج علاقات مؤقتة كبيرة.

مثال 4: (عن ترتيب الدمج)

ليكن التعبير:

$$\Pi_{customer_name} ((\sigma_{branch_city = "Brooklyn"}(branch)) \bowtie (account \bowtie depositor))$$

خيار 1 (غير مفضل):

تنفيذ:

account $\bowtie$ depositor

هذا الربط:

- يحتوي على معظم حسابات البنك
- ينتج علاقة كبيرة جداً

ثم ربطها مع:

$\sigma_{branch_city = "Brooklyn"}(branch)$

← كلفة عالية + مساحة تخزين مؤقتة كبيرة.

خيار 2 (المفضل):

تنفيذ:

$\sigma_{branch_city = "Brooklyn"}(branch) \bowtie account$

- عدد فروع Brooklyn قليل.
- ينتج علاقة صغيرة نسبياً.

ثم نربط الناتج مع: depositor

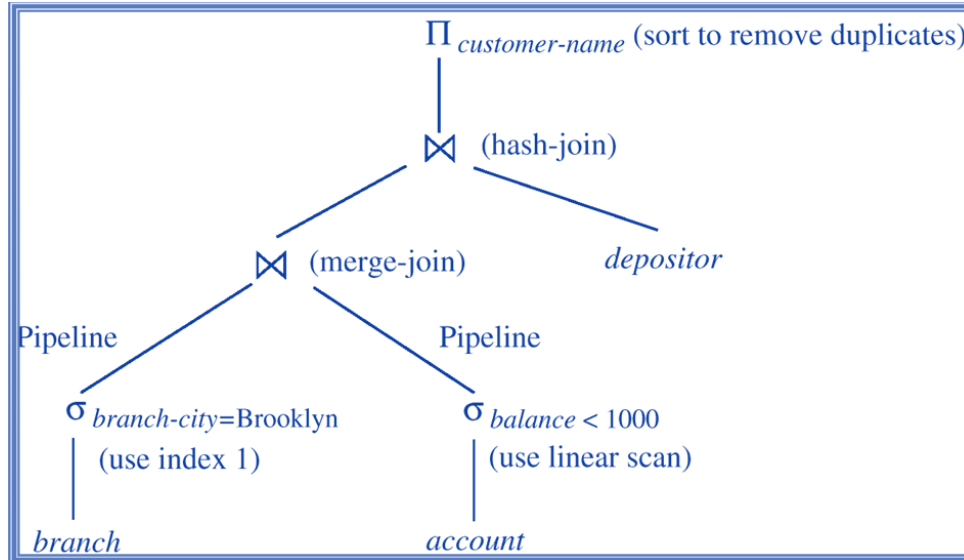
← علاقة مؤقتة أصغر + تحسين كبير في الأداء

تقييم مخطط التنفيذ (Evaluation of an Execution Plan)

مخطط التنفيذ يوضح بالتحديد كيف سيتم تنفيذ الاستعلام فعلياً داخل نظام إدارة قواعد البيانات، ويشمل تقييمه ما يلي:

1. تقييم الخوارزميات المستخدمة لكل عملية
2. تقييم ترتيب تنفيذ العمليات
3. تقييم طريقة تنسيق العمليات معاً

مثال على Evaluation Plan يبين الخوارزميات المستخدمة لكل عملية:



اختيار مخطط التقييم (Choosing an Evaluation Plan)

ما المقصود باختيار مخطط التقييم؟

هو عملية اختيار أفضل خطة تنفيذ للاستعلام من بين عدة مخططات ممكنة، بحيث تكون أقل كلفة إجمالية.

ملاحظة مهمة جداً:

لا يجوز تقييم كل عملية بشكل منفصل لأن:

- الخوارزمية الأقل كلفة لعملية معينة قد لا تعطي أقل كلفة للاستعلام ككل

مثال توضيحي:

لنفترض لدينا استعلام يجمع بيانات من جدولين ثم يقوم بتجميع النتائج:

```

SELECT deptno, COUNT(*)
FROM emp
JOIN dept ON emp.deptno = dept.deptno
GROUP BY deptno;
  
```

■ خيارات Join:

- Hash Join: سريع في خطوة الانضمام، لكن الناتج غير مرتب.
- Merge Join: أبطأ قليلاً في الانضمام، لكن الناتج مرتب حسب deptno

■ تأثير ذلك على التجميع (Aggregation):

- إذا استخدمنا Merge Join، التجميع سيكون أسرع لأن البيانات مرتبة مسبقاً.
- إذا استخدمنا Hash Join، التجميع يحتاج لترتيب البيانات أولاً، مما يزيد الكلفة الإجمالية.

النتيجة: رغم أن Hash Join أسرع في خطوة الانضمام فقط، اختيار Merge Join يؤدي إلى كلفة أقل للاستعلام بالكامل.

⚡ Pipelining التنفيذ المتدفق:

- طريقة تنفيذ مناسبة جداً لـ Nested-Loop Join
 - يتم تمرير الصفوف مباشرة من عملية إلى أخرى
 - بدون تخزين نتائج وسيطة على القرص
- ← يقلل الذاكرة و I/O

⚡ طرق اختيار مخطط التقييم:

1. الاختيار المبني على الكلفة (Cost-Based Selection):

يتم:

- توليد جميع مخططات التنفيذ الممكنة
- حساب كلفة كل مخطط
- اختيار المخطط ذو أقل كلفة

هي أدق طريقة لكنها مكلفة حسابياً

2. الاختيار المبني على الخبرة (Heuristics):

يعتمد على:

- قواعد وخبرات سابقة
- تجارب عملية

مثل:

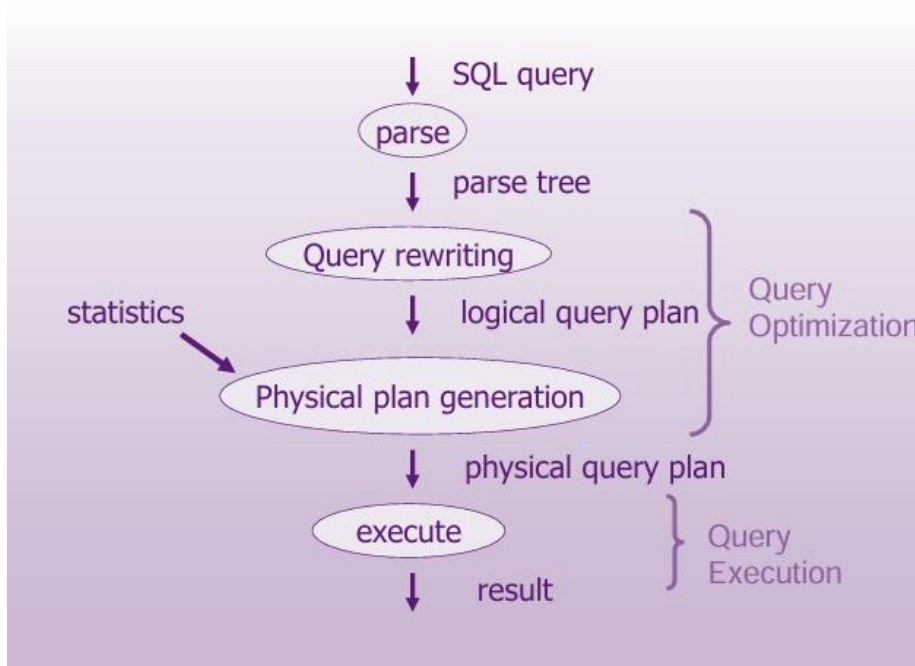
- نقذ Selection مبكراً
- اربط العلاقات الصغيرة أولاً

سريع ولكن قد لا يعطي الحل الأمثل دائماً



ليس كل تأخير حرماً، أحياناً هو حماية

Overview of Query Processing



يشرح المخطط كيف يتعامل نظام إدارة قواعد البيانات مع استعلام SQL خطوة بخطوة:

أولاً: مراحل معالجة الاستعلام Query Processing:

1. الاستعلام النصي – SQL query:

هي الجملة التي يكتبها المستخدم.

2. التحليل النحوي – parse:

يقوم النظام بقراءة الاستعلام والتأكد من:

- صحة الصياغة
- وجود الجداول والأعمدة
- عدم وجود أخطاء لغوية

3. شجرة التحليل – parse tree:

بعد التحليل، يتم تحويل الاستعلام إلى شجرة تمثل بنيته المنطقية.

هذه الشجرة تظهر:

- عمليات التصفية
- عمليات الربط
- الأعمدة المطلوبة
- ترتيب التنفيذ المنطقي

4. إعادة كتابة الاستعلام – Query rewriting:

في هذه المرحلة، يقوم النظام بتحويل الاستعلام إلى صيغة مكافئة ولكن أكثر كفاءة باستخدام قواعد الجبر العلاقي.

5. الإحصاءات – statistics

يعتمد النظام على معلومات عن:

- عدد الصفوف في كل جدول
- حجم الصف
- عدد القيم المميزة
- وجود الفهارس

هذه الإحصاءات تساعد في اختيار الخطة الأفضل.

6. الخطة المنطقية – logical query plan

هي تمثيل للاستعلام باستخدام عمليات الجبر العلاقي مثل:

- selection
- projection
- join

الخطة المنطقية لا تحدد كيف سيتم تنفيذ العمليات، بل ما هي العمليات المطلوبة.

ثانياً: مراحل تحسين الاستعلام Query Optimization:

7. توليد الخطة الفيزيائية – Physical plan generation

هنا يبدأ النظام باختيار:

- خوارزميات التنفيذ مثل hash join أو nested-loop join
- الفهارس المناسبة
- ترتيب العمليات

8. الخطة الفيزيائية – Physical query plan

هذه هي الخطة النهائية التي ستنفذ فعلياً على الملفات، تشمل:

- نوع كل عملية
- ترتيب التنفيذ
- استخدام الفهارس
- طريقة الوصول للبيانات

ثالثاً: تنفيذ الاستعلام Query Execution:

9. التنفيذ – execute

يقوم النظام بتنفيذ الخطة الفيزيائية:

- 1. قراءة البيانات
- 2. تطبيق الشروط
- 3. تنفيذ الربط
- 4. إنتاج النتائج

10. النتيجة – result

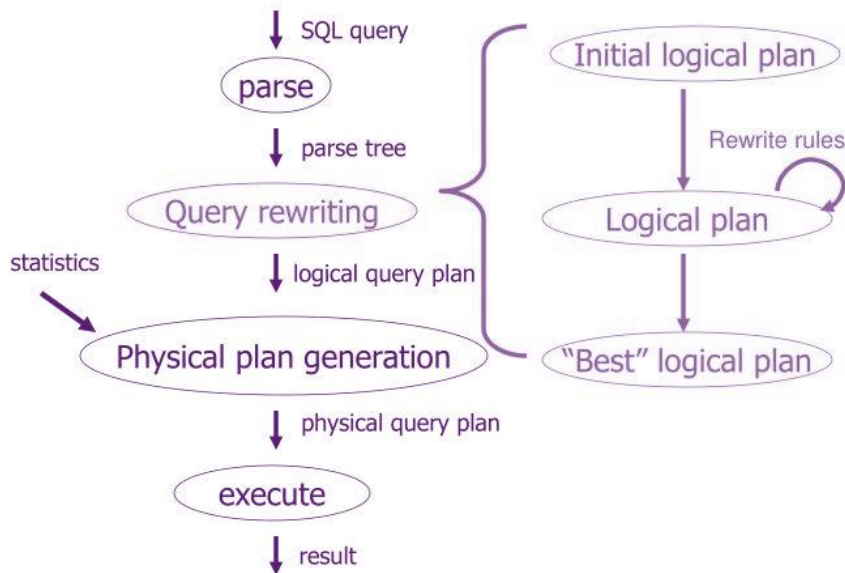
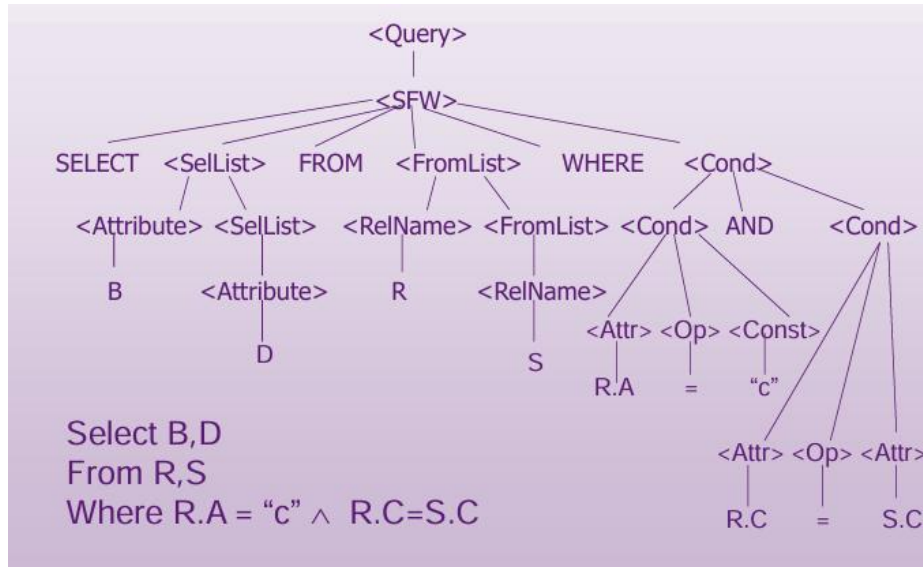
النتيجة النهائية التي يحصل عليها المستخدم.

شجرة التحويل parse tree للاستعلام التالي:

Select B, D

From R, S

Where R.A = "c" AND R.C = S.C



- يوضح المخطط مرحلة ال Query Rewriting حيث تبدأ بمخطط منطقي أولي (initial logical plan)، ثم تطبق قواعد التكافؤ بشكل متكرر (rewrite rules) لتوليد مخططات منطقية متكافئة (logical plan)، ومنها يتم اختيار أفضل مخطط منطقي (best logical plan) ليحوّل لاحقاً إلى مخطط فيزيائي
- السهم (rewrite rules) الموجود على logical plan يعني:
 - تطبيق Rewrite Rules / Equivalence Rules

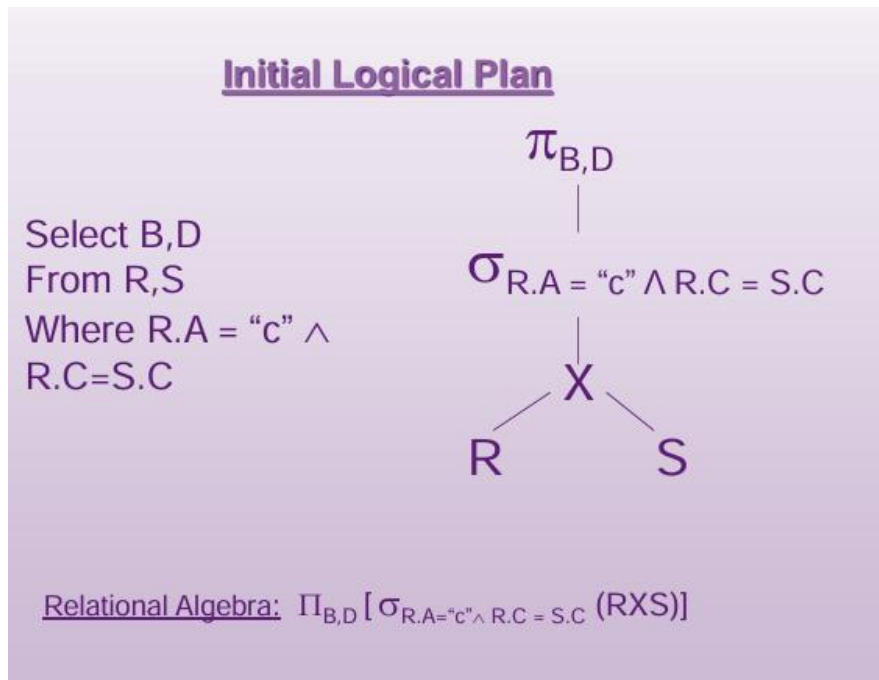
مثل:

- Pushing Selections
- Pushing Projections
- Join Reordering

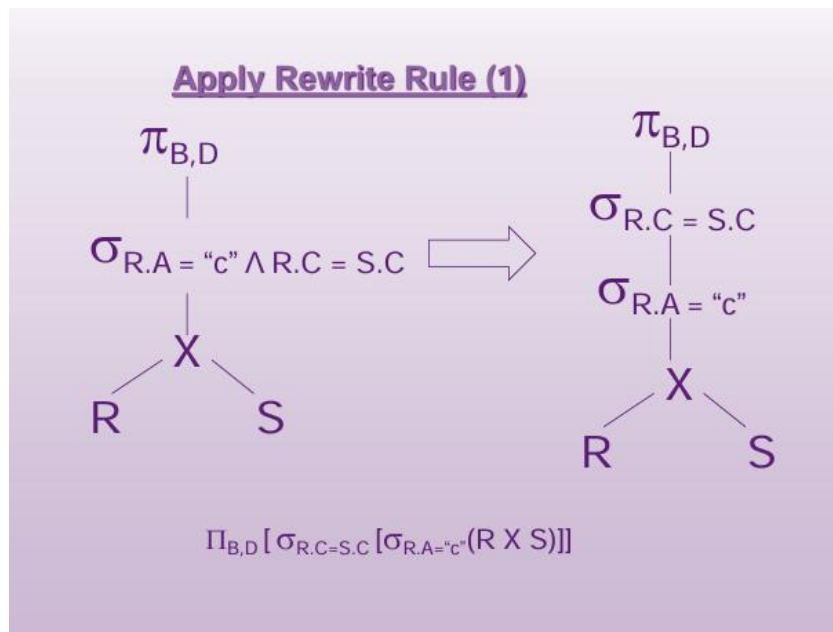
حيث كل تطبيق لقاعدة:

- يحوّل المخطط إلى مخطط منطقي جديد
- لكنه مكافئ منطقيًا (نفس النتيجة)

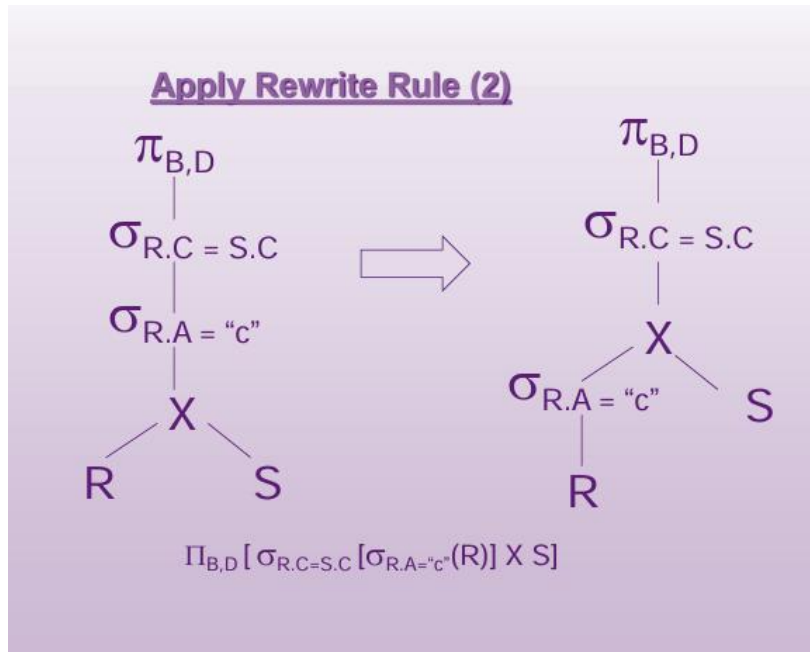
مثال على المخطط المنطقي الأولي للتعليلة نفسها:



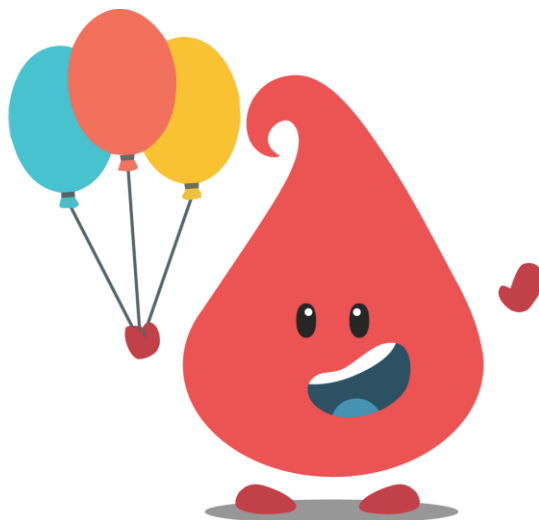
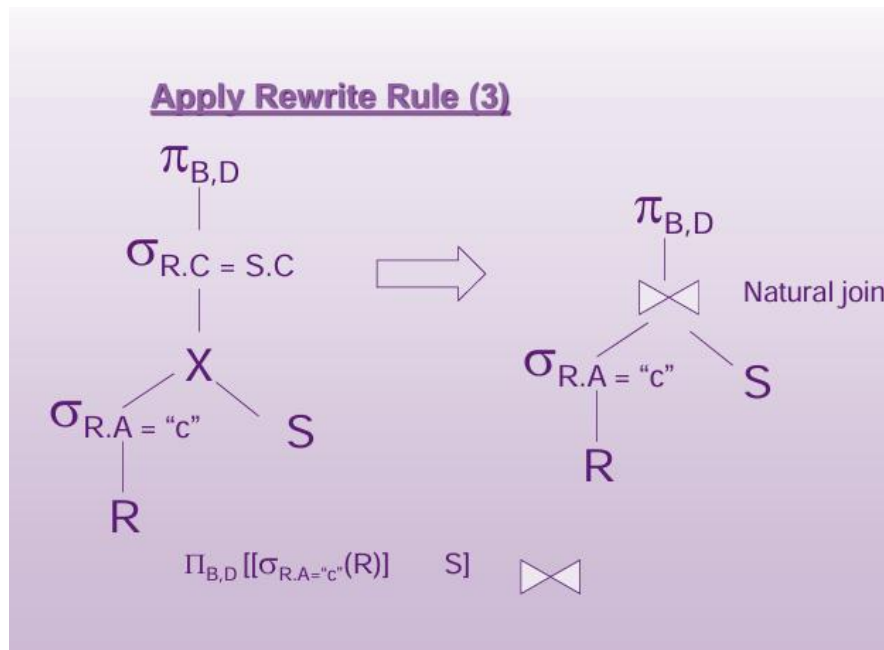
ومن ثم تطبيق قواعد التكافؤ أول مرة:

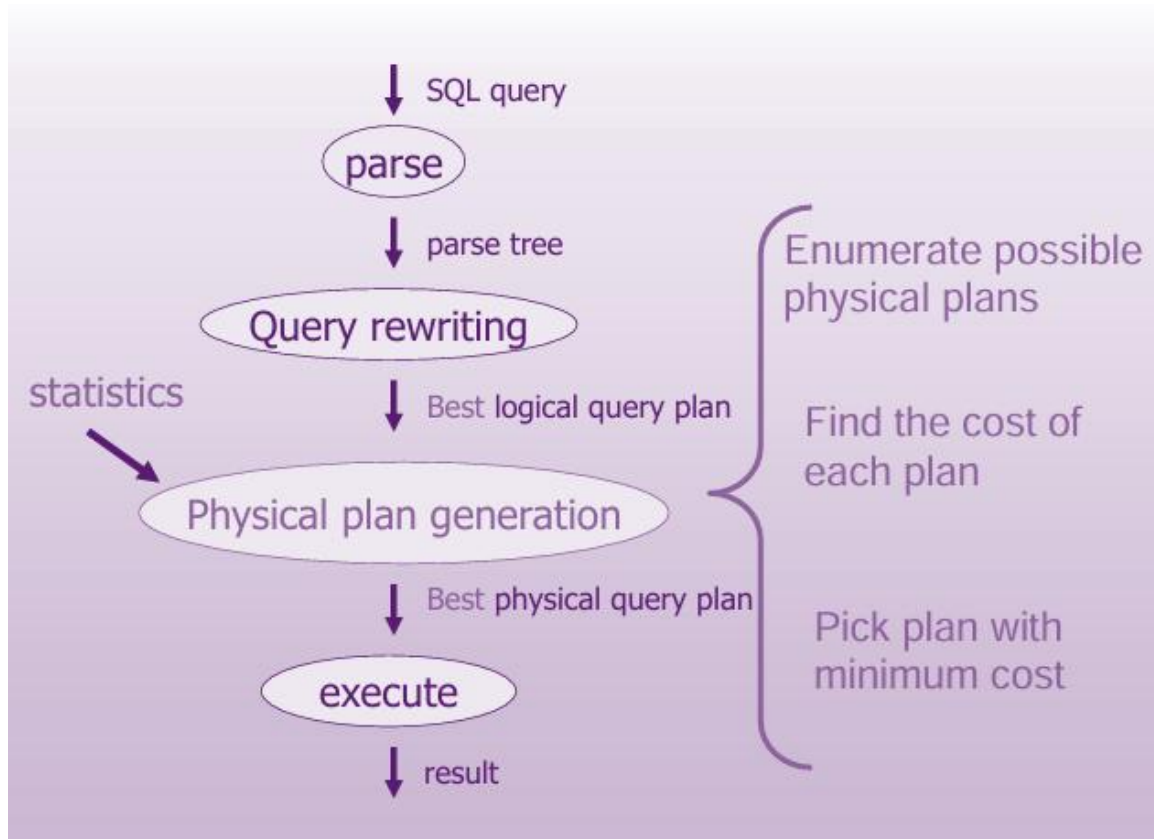


ومرة أخرى (نتيجة مكافئة أخرى):



ومرة ثالثة:





يوضح المخطط هنا مرحلة Physical Plan Generation:

بعد الانتهاء من Query Rewriting واختيار Best Logical Query Plan، ننتقل إلى مرحلة جديدة هي:

Physical Plan Generation (توليد مخطط التنفيذ الفيزيائي)

في هذه المرحلة لا نغيّر منطق الاستعلام، بل نحدد كيف سينفذ فعليًا.

ما الذي يحدث في هذه المرحلة؟ (كما في الشكل)

1. Enumerate possible physical plans

توليد جميع مخططات التنفيذ الفيزيائية الممكنة

▪ لكل عملية منطقية (Join, Selection, Projection):

• يتم توليد عدة بدائل تنفيذ

مثال:

▪ Join يمكن تنفيذه بـ:

• Nested-Loop Join

• Hash Join

• Merge Join

▪ Selection يمكن تنفيذها بـ:

• Full Table Scan

• Index Scan

ومنّه ينتج لدينا عدة Physical Plans للاستعلام نفسه.

2. Find the cost of each plan

حساب كلفة كل مخطط تنفيذ:

يتم حساب الكلفة اعتماداً على:

- Statistics الإحصائيات (موضحة في الشكل)
- الكلفة تشمل:
 - I/O
 - CPU
 - الذاكرة

هذه هي مرحلة Cost-Based Optimization

3. Pick plan with minimum cost

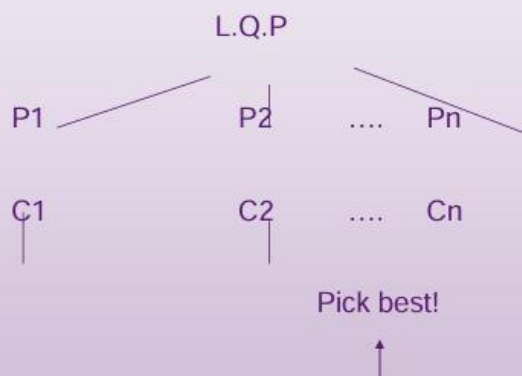
اختيار مخطط التنفيذ الأقل كلفة:

- يتم مقارنة جميع المخططات الفيزيائية.
- اختيار:
 - Best Physical Query Plan
 - أي المخطط ذو أقل كلفة كلية متوقعة
- وفي النهاية يتم:
 - تنفيذ المخطط الفيزيائي المختار
 - إنتاج النتيجة النهائية

ملاحظة:

Query Rewriting = تحسين منطقي (ماذا ننقذ)

Physical Plan Generation = تحسين فيزيائي (كيف ننقذ)

Physical Plan Enumeration and Costing

هذا الشكل يوضح ما الذي يحدث داخل مرحلة Physical Plan Generation بالتحديد:
حيث:

LQP هي اختصار ل Logical Query Plan و هو **أفضل مخطط منطقي** ناتج من مرحلة Query Rewriting
 $P_1, P_2, \dots, P_n$ تمثل:

Physical Plans (مخططات تنفيذ فيزيائية)

من نفس ال Logical Query Plan يتم توليد عدة مخططات تنفيذ فيزيائية مثلا:

• P_1 : استخدام Hash Join + Index Scan

• P_2 : استخدام Merge Join + Full Scan

• P_n : تركيبات أخرى ممكنة

جميعها:

• تنفذ نفس الاستعلام

• تعطي نفس النتيجة

• لكن تختلف في طريقة التنفيذ

وهذه هي الخطوة التي تسمى Physical Plan Enumeration (تعداد مخططات التنفيذ الفيزيائية)

$C_1, C_2, \dots, C_n$ تمثل:

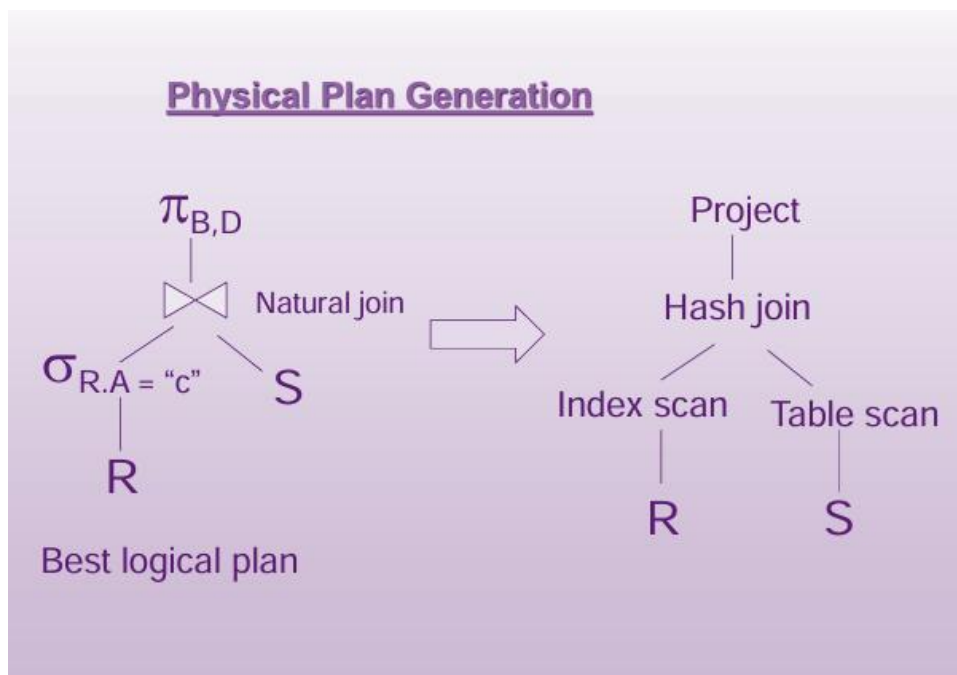
Cost of each plan (كلفة كل مخطط)

• لكل مخطط فيزيائي P_i يتم حساب كلفته C_i

وفي النهاية! Pick best! اختيار أفضل مخطط، حيث تتم مقارنة:

• $C_1, C_2, \dots, C_n$ واختيار best physical plan

هنا نلاحظ التحويل من logical plan الى physical plan (من ماذا سننفذ الى كيف سننفذ)



مثال تعليمي يشرح كيف تتم معالجة استعلام SQL يحتوي على Subquery غير مترابط (Uncorrelated Subquery)، وكيف يتحول هذا الاستعلام عبر المراحل التي تحدثنا عنها سابقاً وصولاً إلى أفضل خطة منطقية:

1. الاستعلام الأصلي (SQL Query)

الهدف: إيجاد الأفلام التي شارك فيها ممثلون وُلدوا في سنة 1960.
الجدول:

MovieStar(name, address, gender, birthdate)

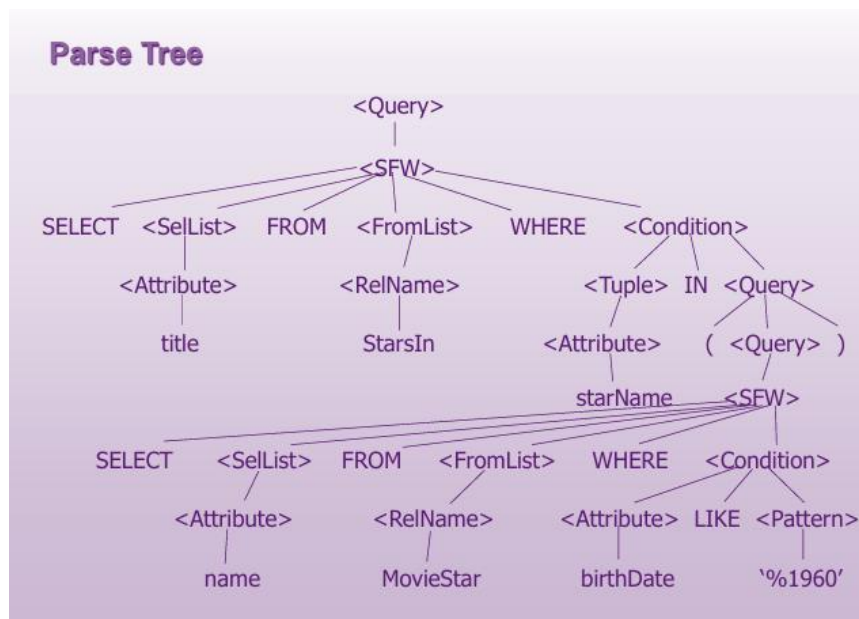
StarsIn(title, year, starName)

الاستعلام:

```
SELECT title
FROM StarsIn
WHERE starName IN (
    SELECT name
    FROM MovieStar
    WHERE birthdate LIKE '%1960'
);
```

هذا استعلام يحتوي على Subquery غير مترابط لأن الجزء الداخلي لا يعتمد على الصفوف الخارجية.

2. مرحلة التحليل (Parse) وبناء شجرة التحليل (Parse Tree)



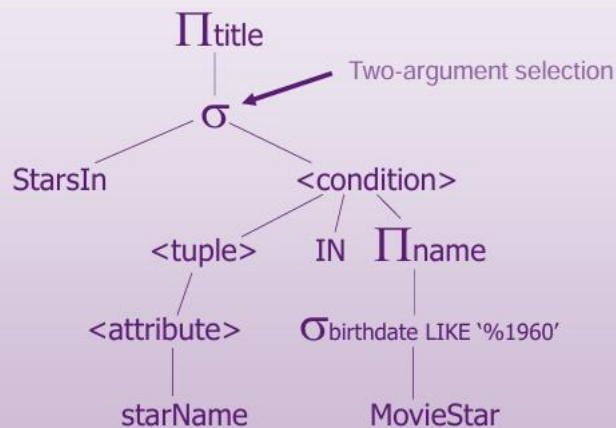
3. توليد الجبر العلاقي (Relational Algebra Generation)

يتم تحويل الاستعلام إلى تعبيرات جبر علاقي:

- إسقاط: π title
- اختيار σ
- الشرط IN يتحول إلى Two-argument selection
- الاستعلام الداخلي يتحول إلى: π name (σ birthdate LIKE '%1960' (MovieStar))

الصيغة الأولية:

Generating Relational Algebra



4. قاعدة إعادة الكتابة الخاصة بال IN (Rewrite Rule)

يوضح الشكل التالي تطبيق قاعدة هامة وهي قاعدة Two-argument selection:

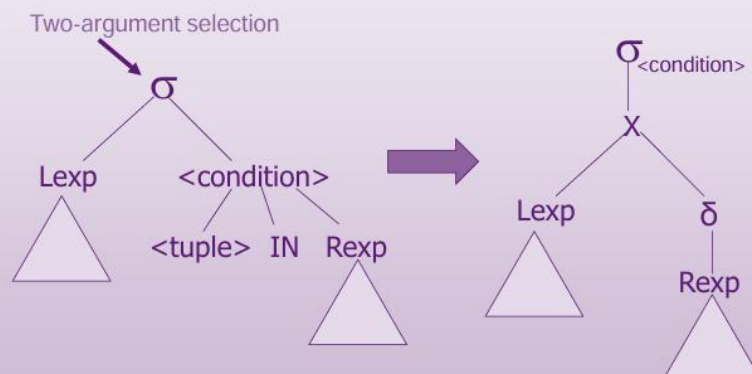
إذا كان لدينا: σ (Lexp <tuple> IN Rexp)

فإنه يُعاد كتابته إلى: σ (condition) (Lexp $\times$ δ (Rexp))

حيث:

■ δ تعني إزالة التكرار (duplicate elimination)

■ يتم تحويل IN إلى Join على الشرط المناسب

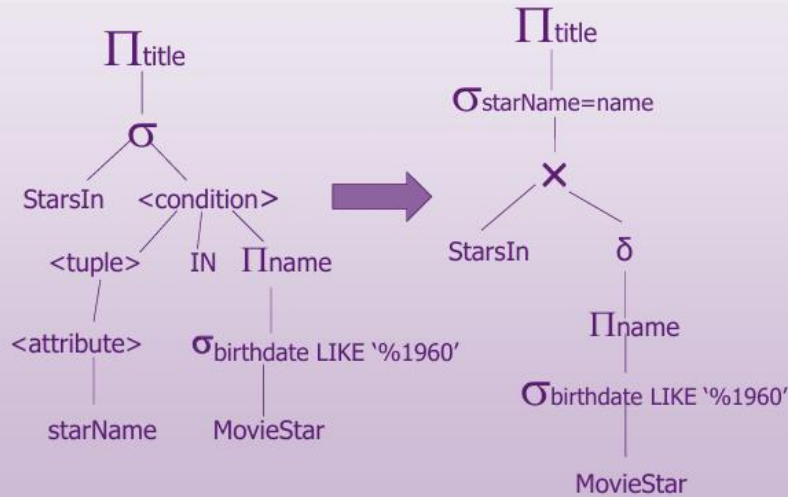
Rewrite Rule for Two-argument Selection with Conditions Involving IN

5. تطبيق قاعدة إعادة الكتابة على الاستعلام

أي:

- نحول IN إلى Join
- نزيل التكرار من أسماء الممثلين
- نربط StarsIn مع MovieStar على الشرط $starName = name$

Applying the Rewrite Rule



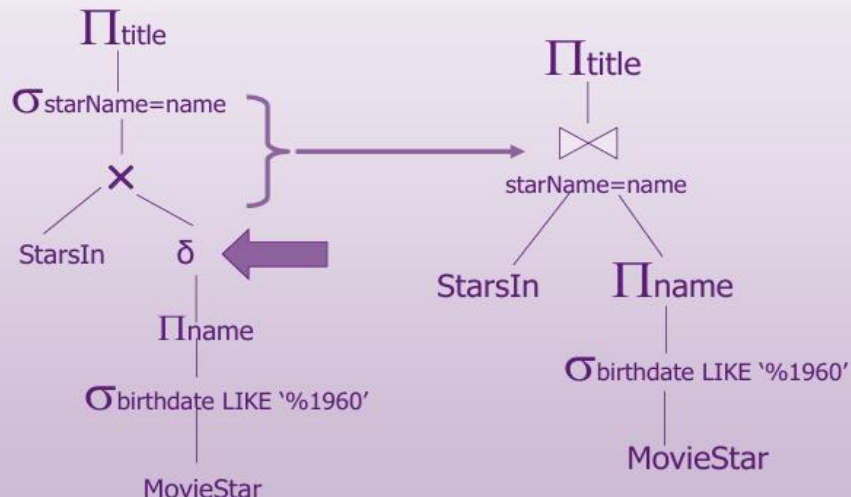
6. تحسين الخطة المنطقية (Improving the Logical Plan):

يبين الشكل استبدال Cartesian Product + Selection بـ Join مباشر، الجهة اليمنى تمثل الخطة المحسنة حيث:

تم دفع العمليات للأسفل (Push Down)

- الإسقاط π name يبقى قريباً من MovieStar
- الاختيار σ birthdate LIKE '%1960' يُنفذ مبكراً
- هذا يقلل عدد الصفوف قبل الربط

Improving the Logical Query Plan



الاختزال المرتكز على الكلفة (Cost-Based Optimization)

الفكرة الأساسية هنا هي إيجاد أفضل ترتيب لعمليات الدمج (JOIN) بين العلاقات $r_1, r_2, \dots, r_n$ بحيث تكون كلفة التنفيذ أقل ما يمكن.

لماذا المشكلة صعبة؟

عدد الطرق الممكنة لترتيب عمليات الدمج كبير جداً ويزداد بشكل انفجاري مع زيادة عدد العلاقات:

- عند $n=3$ ، عدد الترتيبات = 12
- عند $n=5$ ، عدد الترتيبات = 1680
- عند $n=7$ ، عدد الترتيبات حوالي 665,280
- عند $n=10$ ، عدد الترتيبات أكثر من 176 مليار ترتيب

تحسب الترتيبات بالصيغة: $(n-1)! / (2(n-1))!$
لذلك لا يمكن تجربة كل الاحتمالات عملياً.

الحل: البرمجة الديناميكية (Dynamic Programming)

بدلاً من حساب تكلفة كل خطة ممكنة، نستخدم البرمجة الديناميكية لتقليل الحسابات.
الفكرة:

- نحسب أفضل خطة لأي مجموعة جزئية من العلاقات
- نخزن النتيجة
- نستخدمها لاحقاً عند بناء خطط أكبر

مثال:

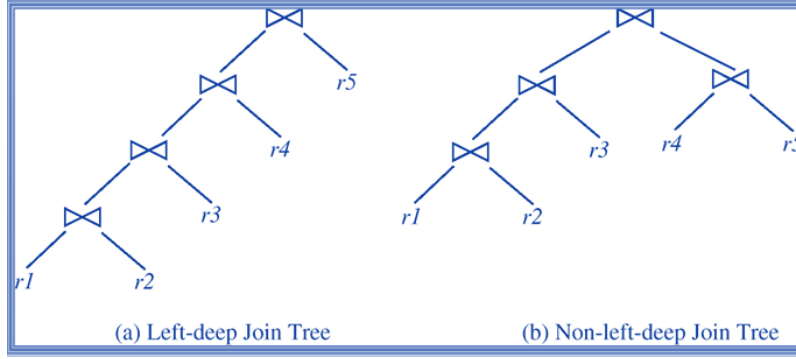
إذا حسبنا أفضل خطة لربط $\{R1, R2\}$ فلا داعي لإعادة الحساب عند ربط $\{R1, R2, R3\}$
← هذا يقلل الحسابات بشكل هائل.

تحسين إضافي: استخدام أشجار الدمج من النوع Left-Deep

لتبسيط البحث أكثر:

- يتم تقييد الخطط إلى Left-deep join trees
- في هذا النوع:
 - الجزء الأيمن من عملية الدمج يكون علاقة أصلية فقط
 - لا تستخدم نتائج دمج وسيطة في الجهة اليمنى

الضغط لا يصنع الماس فحسب، بل يصنع الأبطال



مبدأ الأمثلية (Principle of Optimality)

يندرج مبدأ الأمثلية ضمن حلول مشكلة ترتيب عمليات الدمج، حيث يسمح بتطبيق البرمجة الديناميكية في الاختزال المرتكز على الكلفة، وذلك من خلال افتراض أن الخطة المثالية لأي مجموعة علاقات:

يجب أن تحتوي فقط على خطط جزئية مثالية، مما يقلل عدد خطط الدمج التي يجب فحصها

ينص مبدأ الأمثلية على أن:

أي خطة تنفيذ مثالية لاستعلام ما، يجب أن تكون جميع أجزائها الجزئية خططاً مثالية أيضاً.

أي:

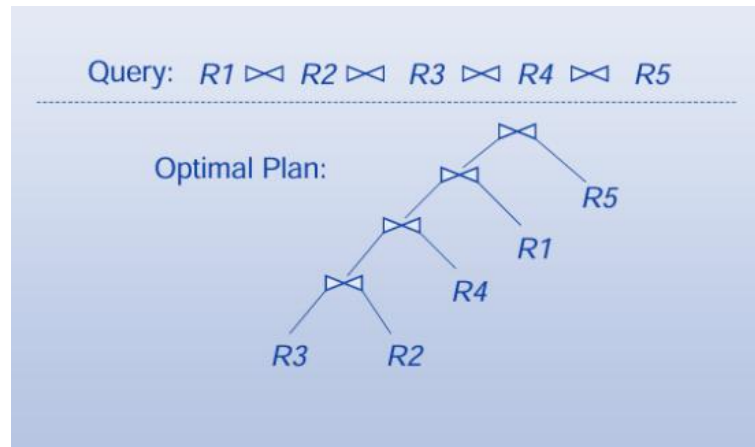
- إذا كان ترتيب دمج العلاقات R_1, R_2, R_3, R_4, R_5 هو الترتيب الأمثل
- فإن أي جزء فرعي منه (مثل دمج R_3, R_2, R_4, R_1) لابد أن يكون هو الآخر ترتيباً أمثل لدمج هذه العلاقات.

لماذا هذا المبدأ مهم؟

- يسمح لنا باستبعاد عدد هائل من الخطط غير المثالية.
- أي خطة تحتوي جزءاً فرعياً غير أمثل لا يمكن أن تكون خطة كلية مثالية.
- هذا يؤدي إلى تقليل عدد الخطط التي يجب فحصها بشكل كبير وإزالة ما يقارب $(n - 3)!$ خطة من الاعتبار

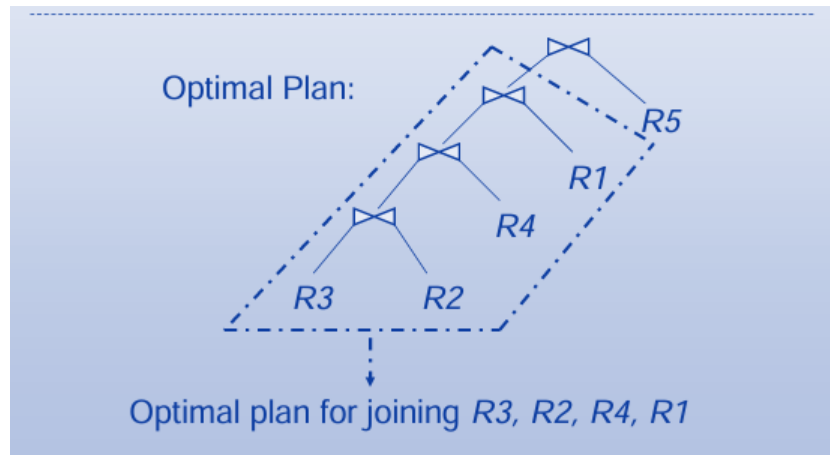
مثال:

نجد هنا الخطة المثلى ل $R_1 R_2 R_3 R_4 R_5$

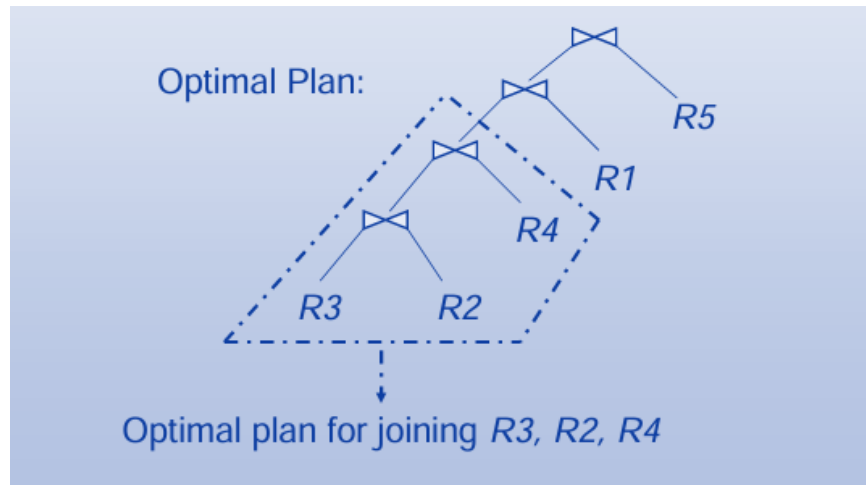


الإرادة تحطم القيود التي تفرضها الحياة

الخطة المثلى لربط $\{R3, R2, R4, R1\}$ هي خطة جزئية من الخطة السابقة:



وبالمثل الخطة المثلى لربط $\{R3, R2, R4\}$ هي خطة جزئية من الخطة السابقة:



نلاحظ أن كل جزء من الخطة الكاملة هو أيضاً أفضل خطة جزئية.

استغلال مبدأ الأمثلية (Exploiting Principle of Optimality)

إذا وجدنا أن:

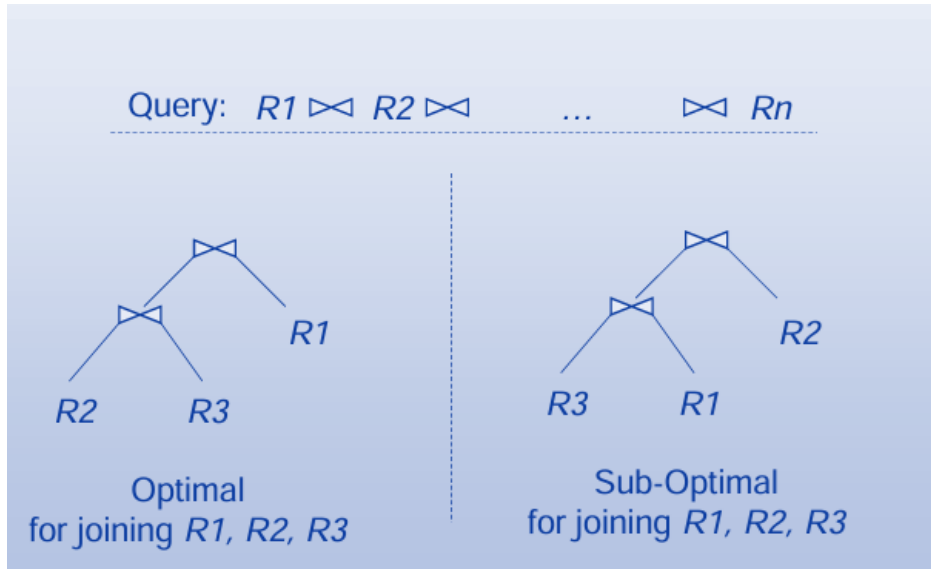
■ ترتيب دمج جزئي مثل $R1, R2, R3$ غير أمثل.

فإن:

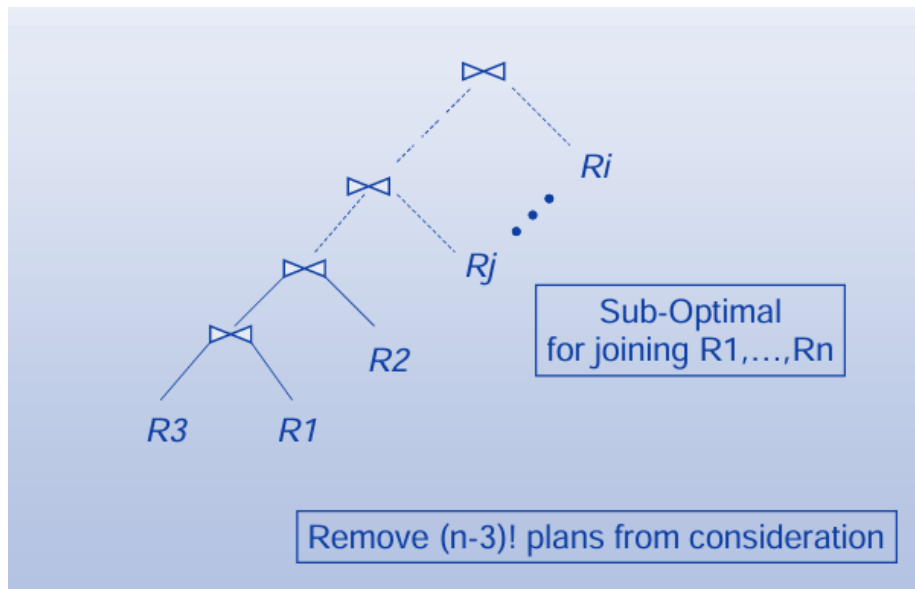
■ أي خطة أكبر تستخدم هذا الترتيب الجزئي $(R1, R2, R3, \dots, Rn)$ تكن مرفوضة مباشرة.

إذا كانت خطة لربط $\{R1, R2, R3\}$ غير مثالية:

ما يصنع الفرق هو الإصرار رغم كل الصعاب



فلا يمكن أن تكون جزءاً من خطة مثالية لربط $\{R1, R2, R3, \dots, Rn\}$:



وبالتالي يمكن حذف $(n-3)!$ من الخطط دون الحاجة لحسابها.

خوارزمية Selinger

هذه الخوارزمية هي أشهر خوارزمية لتحسين الاستعلامات في قواعد البيانات تستخدمها (Oracle و PostgreSQL وغيرها)..

تعتمد خوارزمية Selinger على:

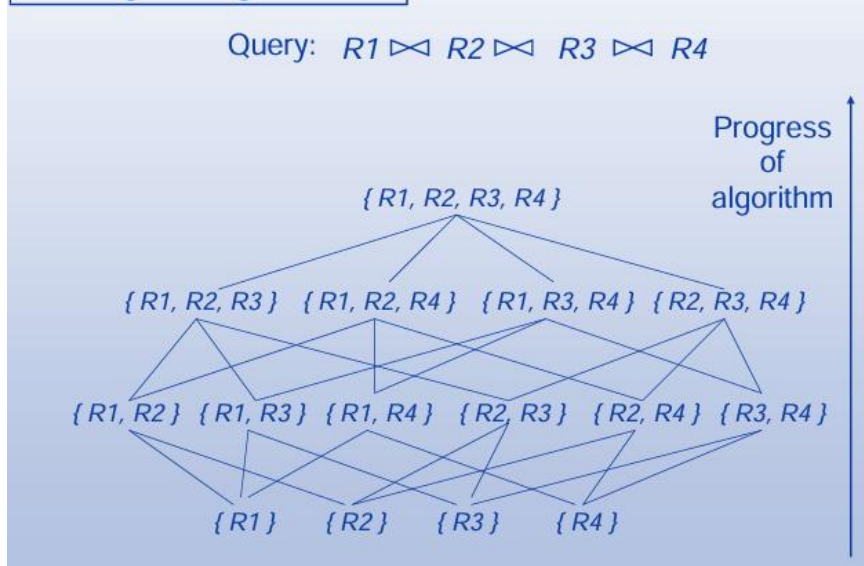
- البرمجة الديناميكية.
- مبدأ الأمثلة
- البحث فقط ضمن Left-Deep Join Trees



فكرة الخوارزمية:

1. تبدأ بالعلاقات المفردة
2. تحسب أفضل دمج لكل مجموعة جزئية من العلاقات
3. تخزن أقل كلفة لكل مجموعة
4. تبني الحل النهائي من الحلول الجزئية المثالية

سنوضح الخطوات فيما يلي:

Selinger Algorithm:

نلاحظ من المخطط أن الخوارزمية:

1. تبدأ بالمجموعات ذات العنصر الواحد:

$\{R1\}, \{R2\}, \{R3\}, \{R4\}$

2. ثم مجموعات من عنصرين:

إلخ $\{R1, R2\}, \{R1, R3\}, \dots$

3. ثم مجموعات من ثلاثة:

$\{R1, R2, R3\}, \{R1, R2, R4\}, \dots$

4. ثم المجموعة الكاملة:

$\{R1, R2, R3, R4\}$

في كل خطوة:

1. نحسب $OPT(S)$ وهي تكلفة أفضل خطة للمجموعة S
2. نحسب $T(S)$ وهو عدد الصفوف الناتجة عن JOIN المجموعة S

$OPT(\{R1, R2, R3\})$:

Cost of optimal plan to join $R1, R2, R3$

$T(\{R1, R2, R3\})$:

Number of tuples in $R1 \bowtie R2 \bowtie R3$

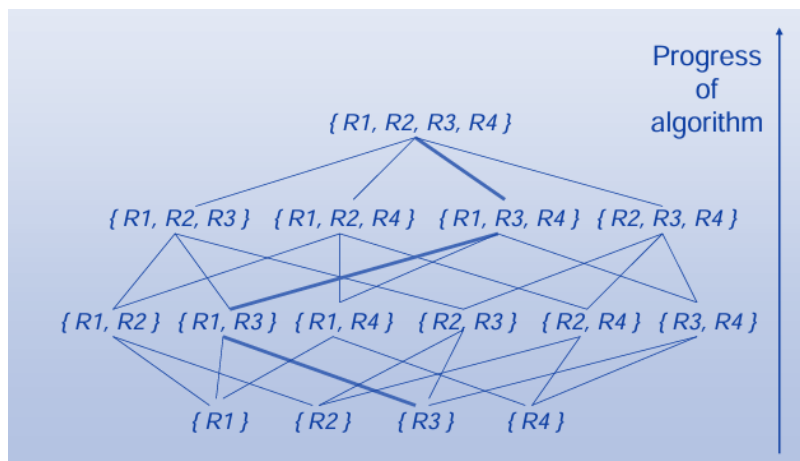
3. لحساب $OPT(S)$ نستخدم المعادلة:

$$OPT(\{R1, R2, R3\}) = \min \begin{cases} OPT(\{R1, R2\}) + T(\{R1, R2\}) + T(R3) \\ OPT(\{R2, R3\}) + T(\{R2, R3\}) + T(R1) \\ OPT(\{R1, R3\}) + T(\{R1, R3\}) + T(R2) \end{cases}$$

ملاحظة:

هذه المعادلة صحيحة فقط مع نموذج كلفة بسيط.

في النهاية يتم بناء الحل المثالي من الحلول الجزئية المثالية:



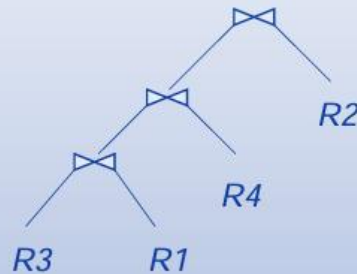
النجاح يحتاج إلى الإصرار وليس الحظ



Selinger Algorithm:

Query: $R1 \bowtie R2 \bowtie R3 \bowtie R4$

Optimal plan:



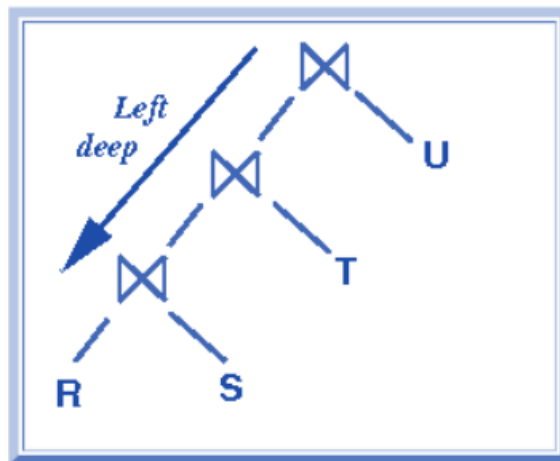
وهذه هي الخطة التي أعطت أقل تكلفة حسب نموذج التكلفة المستخدم.

كيف تُستخدم البرمجة الديناميكية لإيجاد أفضل شجرة دمج من النوع Left-Deep لمجموعة من n علاقات؟

الفكرة الأساسية:

في Left-Deep Join Tree:

- الطرف الأيمن من كل عملية دمج هو علاقة واحدة فقط
- الطرف الأيسر هو ناتج دمج سابق



لايجاد أفضل شجرة دمج لمجموعة علاقات S تحوي n علاقات:

- نفترض أن لدينا أفضل خطة تنفيذ لكل مجموعة جزئية غير فارغة $S1 \subset S$
- تكون الخطط الممكنة على الشكل:

$$S1 \bowtie (S - S1)$$

■ نستخدم:

- الحساب العودي
- وتخزين الكلفة للمجموعات الجزئية "من S لايجاد كلفة كل مخطط"
- اختيار الطريقة الأرخص من بين $2^n - 1$

عدد البدائل التي نحتاج فحصها يصبح $2^n - 1$ فقط نختار منها الأرخص

بدل العدد الهائل لجميع الترتيبات الممكنة

حيث n: هو عدد الطرق الممكنة لربط علاقة من الجانب اليميني في شجرة الدمج يسارية العمق
Left-deep join tree و العلاقات الأخرى من الجانب اليساري

أهمية الترتيب في الاختزال المرتكز على الكلفة

أحياناً يكون ناتج JOIN جزئي مثل $(r1 \bowtie r2 \bowtie r3)$ مُرتباً على صفات معينة، وهذا الترتيب قد يكون مفيداً جداً عند تنفيذ JOIN لاحق مع $r4$ أو $r5$.

هذا ما يسقى interesting sort order

مثال:

إذا كان:

■ $r1 \bowtie r2 \bowtie r3$ ينتج صفوفاً مرتبة على العمود A

■ و $r4$ أيضاً يحتوي على العمود A

فإن merge-join يصبح ممكناً دون الحاجة إلى فرز جديد.

لأن merge-join:

■ سريع جداً عندما تكون البيانات مرتبة مسبقاً

■ بطيء إذا احتاج إلى فرز قبل التنفيذ

إذا كان ناتج JOIN جزئي مرتباً مسبقاً، فهذا يوفر تكلفة كبيرة.



لذلك:

لا يُستبعد هذا الدمج رغم كلفته العالية محلياً لأنه أفضل على مستوى الخطة الكلية

ملاحظة: إن حساب ترتيب الدمج الأفضل لكل تعبير جزئي من التعابير الجزئية الممكن استخراجها من التعبير الأساسي المكون من n علاقة غير كافٍ، ويجب استخدامه كعينة للاختيار في خوارزمية البرمجة الديناميكية.

التحسين المعتمد على الخبرة (Heuristic Optimization)

هو أسلوب يعتمد على قواعد جاهزة (Rules) مبنية على خبرة سابقة في تنفيذ الاستعلامات، بهدف:

- تقليل عدد الخطط المحتملة
 - تبسيط شجرة الاستعلام: (القواعد التي تحقق أداء أفضل في التنفيذ)
 - تطبيق الاختيار أولاً (مبكراً)، مما يقلص من عدد الأسطر المتعامل معها لاحقاً.
 - تطبيق الإسقاط مبكراً، مما يقلص من عدد الواصفات.
 - تطبيق عمليات الاختيار والدمج الأكثر تقليص للناتج قبل تنفيذ بقية العمليات المشابهة.
 - الوصول إلى خطة جيدة بسرعة
- حتى لو لم تكن الخطة هي الأفضل من حيث التكلفة، فهي غالباً قريبة من المثالية. هذا الأسلوب يُستخدم عندما يكون حساب التكلفة لكل خطة محتملة مكلفاً جداً.

لماذا نحتاج Heuristics رغم وجود Cost-Based Optimization؟

لأن:

- عدد خطط JOIN المحتملة كبير جداً
- حساب تكلفة كل خطة يحتاج وقتاً كبيراً
- بعض الأنظمة تستخدم فقط التجربة (Heuristics) وبعضها تدمج بين الطريقتين:
 - Heuristics لتقليل عدد الخطط
 - Cost-Based لاختيار الأفضل من بينها

خطوات الاختزال المرتكز على الخبرة (المعرفة الناتجة عن التجربة)

1. تجزئة عمليات الاختيار متعددة الشروط إلى سلسلة من عمليات الاختيار المتعاقبة وحيدة الشرط.
2. إزاحة عمليات الاختيار إلى الأسفل في شجرة الاستعلام لتنفيذها باكراً قدر المستطاع.
3. تنفيذ عمليات الاختيار و الدمج التي تُنتج علاقات أصغر (عدد الأسطر) أولاً.
4. استبدال عملية الجداء الديكارتي المتبعة بشرط اختيار بعملية دمج.
5. إزاحة عملية الإسقاط على قائمة واصفات قدر المستطاع إلى أسفل الشجرة، وإنشاء عمليات إسقاط جديدة في الأماكن التي نحتاج لها.
6. تحديد الأشجار الجزئية التي يمكن أن تُنفذ العمليات فيها بشكل pipelined، وتنفيذها باستخدام pipelining .

بنية مُحسّن الاستعلام في بعض الأنظمة المعروفة



نظام R:

- يعتمد فقط على Left-Deep Join Orders
- الهدف:
 - تقليل التعقيد
 - توليد خطط تنفيذ قابلة للتنفيذ بأسلوب أنبوبي (Pipelined Evaluation)
- يستخدم الخبرة لدفع:
 - عمليات الاختيار (Selection)
 - والإسقاط (Projection) إلى أسفل شجرة الاستعلام قدر الإمكان

نظام Sybase:

- في نظام Sybase، يُفترض عادةً أن الصفحة الحاوية على السطر المطلوب موجود في الذاكرة المؤقتة (buffer)، لذلك يبدأ البحث أولاً داخل الـ buffer قبل أي مكان آخر.

أنظمة أخرى:

- تستخدم بعض مختبرات الاستعلام في العديد من الأنظمة أسلوب الاختزال المعتمد على الخبرة، وذلك لتوليد مخططات وصول مختلفة.
- ويعتمد مختزل [نظام Oracle7](#) على الخبرة المرتكزة على طرق الوصول المتاحة للبيانات

Oracle Optimizer

Oracle يدعم طريقتين للاختزال:

1. الاختزال المرتكز على القواعد (Rule-Based Optimization (RBO

- يعتمد على قواعد ثابتة
- يختار:
 - مخطط التنفيذ بناءً على خبرة مسبقة (العمليات المختبرة تجريبياً)، وليس على حساب الكلفة الفعلية

2. الاختزال المرتكز على الكلفة (Cost-Based Optimization (CBO

- يعتمد على:
 - طرق الوصول
 - ترتيب العمليات
- يختار:
 - الخطة ذات أقل كلفة تنفيذ
- مختزل oracle يمكن المبرمج من:

```
SELECT /*+ INDEX(emp indx_sex) */
*
FROM emp
WHERE sex = 'M'
```

هنا نُجبر Oracle على استخدام الفهرس.

أمثلة على قواعد Oracle لتحسين الاستعلام (Heuristic Rules):

Oracle يقوم بتحسين الاستعلام عبر تحويله إلى صيغة مكافئة لكنها أسرع:

1. تحويل LIKE إلى = إذا أمكن:

```
Ename LIKE 'King'
→ Ename = 'King'
```

ملاحظة: هذا التحويل غير ممكن في الحالات التي يتم استخدام الحرفين "-" أو "%" "

2. تحويل IN إلى مجموعة OR (أو العكس حسب الفهارس)

```
Ename IN ('King', 'Jones', 'Foo')
→ Ename='King' OR Ename='Jones' OR Ename='Foo'
```

3. تحويل OR إلى UNION ALL عند وجود فهارس:

إذا كان OR يمنع استخدام الفهرس:

```
job='clerk' OR deptno=20
```

يحوّله Oracle إلى:

```
SELECT * FROM emp WHERE job='clerk'
UNION ALL
SELECT * FROM emp WHERE deptno=20
```

وهذا يسمح باستخدام الفهرس في كل جزء.

لا شيء، يدوم للأبد، فاحرص على اللحظة

4. تحويل Subquery إلى Join:

مثال:

```
SELECT *
FROM account
WHERE customer-name IN
(SELECT customer-name FROM customer);
```

يُحوّل إلى:

```
SELECT account.*
FROM account, customer
WHERE account.customer-name = customer.customer-name;
```

5. تحسين الاستعلامات مع Views:

إذا وُجد View:

- يقوم Oracle بتفكيكه ثم تطبيق الشروط على الجداول الأصلية

مثال:

```
SELECT empno, ename FROM uemp WHERE deptno=20;
```

يُحوّل إلى:

```
SELECT empno, ename FROM emp1 WHERE deptno=20
UNION
SELECT empno, ename FROM emp2 WHERE deptno=20;
```

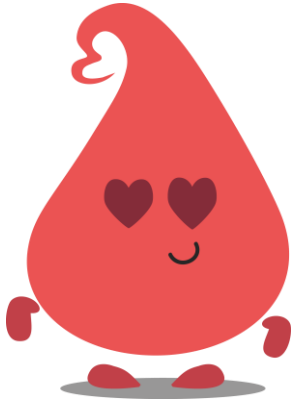
- Uemp هنا View، وليس جدولاً حقيقياً.

- Oracle لا يخزن بيانات ال View نفسها، بل يحتفظ بالاستعلام الذي يعرف ال View.

ما يحدث داخلياً:

1. Oracle يفكك ال View (expansion): أي أنه يضع مكان ال View الاستعلام الأصلي الذي أنشأ ال View.
2. ثم يقوم بتطبيق أي شروط في الاستعلام الخارجي (WHERE deptno=20) على الجداول الأصلية.

الطريق الصعب غالباً يقود لأجمل النتائج



طرق الوصول إلى البيانات في Oracle:

Oracle يدعم عدة Access Paths:

- Full Table Scan: مسح تسلسلي
 - RowID Access: وصول مباشر
 - Index Search: عبر الفهارس
 - Cluster Access: تخزين علاقات مترابطة معاً
 - Hash / Partition Access: باستخدام مفتاح التقطيع
- المُحسِّن يختار الأنسب حسب الكلفة.

الاختزال الدلالي (Semantic Query Optimization)

نوع آخر من الاختزال يعتمد على:

- قيود السلامة
- المعرفة الدلالية في قاعدة البيانات

مثال:

استعلام يبحث عن موظفين يتقاضون راتباً أعلى من رؤسائهم.

إذا كانت قاعدة البيانات تحتوي قيداً يمنع ذلك:

← الاستعلام لا يحتاج تنفيذ أصلاً

← النتيجة معروفة مسبقاً (فارغة)

هذا يوفر:

- وقت التنفيذ
- موارد النظام

ملخص:

النظام	أسلوب التحسين
System R	left-deep trees + heuristics
Sybase	buffer-aware + hybrid heuristics
Oracle RBO	قواعد ثابتة + hints
Oracle CBO	حساب التكلفة + قواعد مساعدة
Semantic Optimization	استخدام القيود لتقليل العمل

The End...